

VISHWAKARMA
UNIVERSITY
Maximising Human Potential

AI-Based Complexity-Aware Delivery Route Optimization System

(RouteOptimaAI)

Student Name: Yug Shah

SRN No: 31241195

Roll No: 49

Year & Program: T.Y. B.Tech CSE (AI&ML)

Division: C

Subject: Design and Analysis of Algorithms (DAA)

Table of Contents

Sr. No.	Content Description
1.	Abstract
2.	Introduction
3.	Objectives
4.	Tools and Technologies Used
5.	Methodology Part 1: Workflow, Data Schema & Greedy Nearest Neighbor
6.	Results
7.	Advantages
8.	Limitations
9.	Future Scope
10.	Conclusion
11.	GitHub Link

1. Abstract

Last-mile delivery route planning is one of the most operationally challenging and computationally expensive components of commercial supply chains. Inefficient manual scheduling creates redundant travel mileage, sub-optimal vehicle capacity utilization, elevated carbon emissions, and frequent delivery time window violations. Mathematically framed as the Capacitated Vehicle Routing Problem with Time Windows (CVRPTW), finding optimal multi-vehicle delivery schedules is strongly NP-hard.

To resolve this challenge, RouteOptimaAI introduces an AI-based, complexity-aware dispatch and decision-support system. Rather than forcing every delivery problem through a single static algorithm, the system dynamically assesses the structural complexity of each routing scenario—quantifying order volume, vehicle capacity tightness, priority ratios, time-window deadlines, and spatial dispersion across Pune, Maharashtra, India. An explainable, 100% local Scikit-Learn Random Forest classifier evaluates these complexity features to recommend the most suitable solver: Greedy Nearest Neighbor for rapid heuristic construction, Local Search (2-Opt and order relocation) for iterative neighborhood improvement, Genetic Algorithm for global population exploration, or a Hybrid Metaheuristic (Genetic + 2-Opt) for complex scenarios. A deterministic safety arbiter guarantees that physical vehicle payload capacities are respected and no orders are omitted or duplicated.

RouteOptimaAI integrates a modern React 19 single-page dashboard with an interactive Leaflet mapping engine, a high-performance Python FastAPI backend, SQLite/PostgreSQL persistence, and live bi-directional WebSocket telemetry (`/ws/live`). Routing is powered by OpenStreetMap Nominatim geocoding, OSRM road network polylines, and intelligent caching with road-curvature fallbacks. Empirical benchmarking across scenarios from 10 to 80 orders proves that while Greedy executes instantaneously (< 0.001 s), Local Search and Hybrid eliminate late deliveries entirely (dropping late orders from 17 down to 0 at 80 orders). All 19 automated unit and integration tests pass successfully, confirming system stability and educational rigor.

2. Introduction

2.1 Background

Last-mile logistics accounts for over 40% to 50% of total transportation costs in commercial logistics. In modern urban delivery operations, a fleet of vehicles with finite payload capacities must depart from central depots to deliver packages across dense, traffic-heavy metropolitan areas. Dispatchers face two critical combinatorial challenges: (1) Fleet Clustering (assigning orders to vehicles without exceeding capacities), and (2) Route Sequencing (ordering customer visits to minimize total distance while satisfying delivery time windows).

2.2 Problem Statement

Manual planning fails because the candidate route space grows factorially: for n customers and m vehicles, the search space is O(m^n · n!). Real delivery operations impose competing constraints: vehicle capacities cannot be breached, urgent parcels must be expedited, and time windows must be met. Furthermore, myopic decisions—such as visiting the nearest customer first—frequently strand subsequent deliveries at extreme geographical peripheries, causing long, cross-town return trips with empty vehicles.

2.3 Proposed System

RouteOptimaAI provides an automated, complexity-aware route planning architecture. It combines four distinct Design and Analysis of Algorithms (DAA) solvers, an 8-feature machine learning strategy classifier, OpenStreetMap spatial integration, and an interactive web dashboard with real-time WebSocket vehicle simulation.

2.4 Scope of the Project

The implemented system supports multi-vehicle CVRPTW routing, address geocoding across Pune, four optimization algorithms, live GPS simulation, Before vs After analytics, and comprehensive REST APIs. Live hardware OBD-II GPS tracking and dynamic en-route re-optimization remain future extensions.

2.5 Motivation

This project demonstrates practical synergy between theoretical algorithm design (greedy heuristics, local search, genetic metaheuristics) and applied machine learning (scenario classification), providing significant logistical, educational, and environmental value.

3. Objective

1. Automate Last-Mile Scheduling: Eliminate manual route guesswork with an automated mathematical planning pipeline.
2. Implement 4 DAA Solvers: Construct and benchmark Greedy Nearest Neighbor, Local Search (2-Opt), Genetic Algorithm, and Hybrid Optimizer.
3. Explainable ML Selection: Integrate a 100% local Random Forest model to recommend solvers with human-readable rationale.
4. Enforce Physical Feasibility: Implement a RouteEvaluator checking capacity limits, arrival times, and duplicate order prevention.
5. Automatic Geocoding: Enable plain address import without pre-existing coordinates using Nominatim and a verified Pune directory.
6. Real Road Network Routing: Connect with OSRM to generate genuine turn-by-turn road polyline geometries rather than straight lines.
7. Interactive Web Dashboard: Build a React 19 interface with multi-vehicle colored Leaflet routes and Before vs After analytics.
8. Real-Time Telemetry Simulation: Stream live simulated vehicle coordinates, speeds, and stop completions over WebSockets (/ws/live).
9. Maintain Rigorous Verification: Pass 19 automated Pytest unit and integration tests covering API, ML, geocoding, and solvers.
10. Empirical Benchmarking: Document quantitative runtime, distance, and constraint metrics across 10, 20, 50, and 80 order instances.

4. Tools and Technologies Used

4.1 Frontend Technologies

The user interface is built with React 19 (concurrent rendering, hooks), Vite 8.3.0 (fast ES module bundling), Leaflet 1.9.4 and React-Leaflet 5.0.0 (interactive map rendering, custom SVG markers, multi-vehicle polyline layers), Lucide React (scalable UI iconography), and a custom Vanilla CSS design system (dark palette #0e1117, responsive flex/grid layouts) avoiding bloated external CSS libraries.

4.2 Backend Technologies

The server layer uses Python 3.10+ (tested on Python 3.14), FastAPI 0.115+ (asynchronous REST endpoints, automatic OpenAPI documentation at /docs), Uvicorn 0.30+ (high-concurrency ASGI server), Pydantic v2.8+ (strict request/response data validation), WebSockets 12.0+ (full-duplex persistent telemetry streaming at /ws/live), and HTTPX (asynchronous external HTTP client).

4.3 Machine Learning Technologies

The strategy selection engine uses Scikit-Learn 1.5+ (RandomForestClassifier, 120 estimators, max depth 10, balanced class weights), Pandas 2.2+ and NumPy 1.26+ (data manipulation and array processing), and Joblib (model persistence). The model achieves 85.00% accuracy and 82.54% weighted F1-score across 8 features (order count, vehicles, constraints, average distance, geographic spread, priority ratio, time window ratio, and capacity utilization). It operates 100% locally without external LLM dependencies.

4.4 Database and Storage

Relational storage is implemented via SQLAlchemy 2.0+ ORM with SQLite (delivery.db) for local execution, ready for PostgreSQL in production. Ten database entities persist the logistics state:

Technology / Entity	Responsibility & Purpose	Verified Module / File
SQLite / SQLAlchemy	Relational persistence of fleet, orders, routes, runs	backend/app/database.py
Order & Customer Models	Stores recipient addresses, package weights, time windows	backend/app/models/entities.py
Vehicle & Location Models	Maintains fleet payload limits, assigned routes, live GPS breadcrumbs	backend/app/models/entities.py
Route & RouteStop Models	Persists chronological delivery stops, arrival ETAs, road polylines	backend/app/models/entities.py
OptimizationRun & Cache	Records run diagnostics, complexity scores, OSRM matrix cache	backend/app/models/entities.py

4.5 Routing, Geocoding & Development Tools

Spatial intelligence connects to OpenStreetMap Nominatim for geocoding and OSRM for road network driving paths. An in-memory dictionary of 40+ Pune localities and 20+ PIN codes ensures instant offline address resolution. Pytest 8.0+ executes 19 automated tests; Git manages version control.

5. Methodology

5.1 Overall Workflow

RouteOptimaAI processes deliveries through a structured 10-step pipeline: (1) Dispatcher enters orders or uploads a batch CSV; (2) Backend validates fields, positive weights, and priorities; (3) Geocoding service resolves coordinates via Pune dictionary or Nominatim; (4) Spatial service constructs NxN distance/duration matrix with road curvature adjustments; (5) Complexity service extracts 8 normalized features; (6) Random Forest classifier predicts the optimal solver; (7) Safety arbiter verifies feasibility rules; (8) Designated DAA algorithm optimizes vehicle routes; (9) RouteEvaluator audits costs, time windows, and vehicle capacity; (10) Routes, ETAs, and Before vs After metrics are stored and rendered on the Leaflet map.

5.2 Input Data & Validation

Orders require: order code, customer name, address, area, PIN code, package weight (kg), priority (LOW, NORMAL, HIGH, URGENT), and time windows. Coordinates are automatically geocoded, preventing user burden.

Order ID	Customer Name	Address / Locality	Weight	Priority	Time Window
ORD001	Aarav Sharma	Baner Road, Baner	2.5 kg	HIGH	10:00 – 12:00
ORD002	Priya Deshmukh	Wakad Hinjewadi Rd	1.8 kg	NORMAL	12:00 – 14:00
ORD003	Rohan Kulkarni	Aundh D-Mart, Aundh	3.2 kg	HIGH	14:00 – 16:00
ORD004	Sneha Patil	Kothrud Vanaz Corner	4.0 kg	NORMAL	11:00 – 15:00

5.3 Data Validation & Resilient Preparation

Validation enforces: weight > 0, non-empty addresses, and valid time windows. Unlocated addresses are marked FAILED—the system never invents fake coordinates. Unlocated orders are isolated from the routing pool.

5.4 Greedy Nearest Neighbor Heuristic

Greedy builds tours starting at the depot, iteratively appending the nearest unvisited order whose weight fits vehicle capacity. Perceived distance is weighted by priority multipliers: URGENT (0.70x), HIGH (0.85x), NORMAL (1.0x), and LOW (1.15x), biasing the search to serve critical orders sooner.

Pseudocode: Greedy Nearest Neighbor

```
Algorithm Greedy_CVRPTW(Orders, Vehicles, DistMatrix):
  Unvisited = {1, 2, ..., N}; Routes = [[] for v in Vehicles]
  For v in Vehicles:
    CurrLoad = 0; CurrLoc = DEPOT
    While Unvisited:
      Candidate = argmin_{i in Unvisited} (DistMatrix[CurrLoc][i] * PriorityMultiplier(i))
```

If CurrLoad + Weight(Candidate) <= Capacity(v):
Append Candidate to Routes[v]; Unvisited.remove(Candidate)
CurrLoad += Weight(Candidate); CurrLoc = Location(Candidate)
Else: Break to next vehicle
For each remaining order in Unvisited: Assign to vehicle with minimum overload
*Return Routes // Time Complexity: O(K * N^2), Space Complexity: O(N)*

5.5 Local Search: 2-Opt and Order Relocation

Local Search refines an existing solution through intra-route and inter-route improvements:
1. Intra-Route 2-Opt Edge Inversion: Reverses intermediate sub-tours between stops i and j: NewRoute = (v_0...v_i) + (v_j...v_{i+1}) + (v_{j+1}...v_m). Reconnecting edges eliminates crossed paths whenever CostDelta < -1e-4.
2. Intra-Route 1-Opt Insertion: Relocates single deliveries along the same vehicle tour to improve time window alignment.
3. Inter-Route Relocation: Shifts an order from Vehicle v1 to Vehicle v2 if v2 has surplus payload capacity and the net combined cost delta is negative. Iterates until convergence or budget limit (max 150 iterations).

5.6 Genetic Algorithm (Metaheuristic)

A population-based evolutionary metaheuristic exploring global route ordering combinations:
• Chromosome Representation: Permutation of all N order indices [o_1, o_2, ..., o_N].
• Greedy Capacity Decoder: Decodes permutations by placing each order on the least-loaded vehicle with sufficient capacity.
• Fitness Function: Inversely proportional to objective cost: Fitness = 1.0 / (TotalCost + 1e-4).
• Selection & Elitism: Tournament selection (k=3); preserves top 2 elite chromosomes directly into the next generation.
• Order Crossover (OX): Copies a slice from Parent 1 and fills remaining positions in Parent 2 sequence without duplicates.
• Mutation (Rate = 0.20): 50% probability of Swap Mutation; 50% probability of Inversion Mutation.

5.7 Hybrid Algorithm (Genetic + 2-Opt)

A cooperative metaheuristic combining global exploration with targeted local exploitation. Phase 1 runs the Genetic Algorithm for 80% of the allocated time budget to discover high-quality global clustering. Phase 2 seeds the best chromosome into Local Search for the remaining 20% budget, performing 2-opt edge untangling.

5.8 Machine Learning Complexity-Aware Selection

The ComplexityService calculates a normalized Complexity Score (0–100) combining four weighted factors: (1) Order volume scale (up to 40 pts), (2) Fleet capacity tightness (up to 15 pts), (3) Constraint density (up to 20 pts), and (4) Spatial dispersion (up to 25 pts). Scores < 40 are LOW, 40–70 are MEDIUM, and > 70 are HIGH.

The Random Forest classifier evaluates the 8-feature vector and outputs strategy probabilities. The Safety Arbiter applies deterministic rules: problems with <= 6 orders are forced to GREEDY (avoiding overhead); problems with Score >= 75 and >= 25 orders upgrade to HYBRID.

5.9 Route Evaluation and Mathematical Objective Function

The unified RouteEvaluator scores candidate solutions using a weighted multi-objective function with penalty barriers:

$$\textbf{min Cost} = w_{dist} \cdot Dist_{total} + w_{time} \cdot Time_{travel} + P_{late} \cdot N_{late} + P_{cap} \cdot V_{cap} + P_{unassigned} \cdot N_{unassigned} + P_{dup} \cdot N_{dup}$$

Default parameters: w_dist = 1.0, w_time = 0.2, P_late = 50.0 per late order, P_cap = 1000.0 per kg overloaded, P_unassigned = 5000.0, P_dup = 5000.0, Depot start = 09:00 AM (540 min), Service time = 5.0 min per customer. A solution is marked feasible if and only if V_cap == 0, N_unassigned == 0, and N_dup == 0.

5.10 System Architecture Overview

The system architecture follows a clean multi-tier decoupling: (1) Presentation Layer: React 19 single-page application with Leaflet mapping; (2) Service & API Layer: FastAPI REST endpoints and WebSocket live broadcast; (3) Spatial Engine: OSRM road geometry builder, Nominatim geocoder, and caching layer; (4) Intelligence Engine: Complexity analyzer, Random Forest classifier, and DAA solver suite; (5) Persistence Layer: SQLite/PostgreSQL relational database.

5.11 Time and Space Complexity Analysis

Theoretical asymptotic bounds compared with empirical runtimes at N = 80 orders across 10 vehicles:

Algorithm	Time Complexity	Space Complexity	Parameters Defined	Runtime (N=80)
Greedy Nearest Neighbor	O(K · N^2)	O(N)	N orders, K vehicles	< 0.001 s
Local Search (2-Opt & Reloc)	O(I · K^2 · N^2)	O(N)	I iterations (I <= 150)	2.100 s
Genetic Algorithm	O(G · P · N)	O(P · N)	P population (50), G gen (80)	0.290 s
Hybrid (Genetic + 2-Opt)	O(G·P·N + I·N^2)	O(P · N)	80% time GA, 20% time 2-Opt	2.450 s

Complexity Insights: Greedy offers sub-millisecond execution because it never backtracks, but produces sub-optimal tours. Local Search scales with improvement iterations I. Genetic Algorithm runtime is strictly bounded by generation budget G and population P. Hybrid balances global exploration with local edge inversion, remaining well within operational interactive limits (< 2.5 s for 80 orders).

6. Results

6.1 Functional Results

RouteOptimaAI was verified against 19 automated Pytest unit and integration tests (19 passed, 0 failed). All core operational flows function reliably: CSV import, Pune geocoding, complexity analysis, strategy selection, multi-vehicle routing, Before vs After analytics, and live WebSocket telemetry.

6.2 Algorithm Comparison Matrix

The four implemented algorithms exhibit distinct engineering trade-offs across solution quality, execution time, and constraint handling:

Metric / Dimension	Greedy Nearest Neighbor	Local Search (2-Opt)	Genetic Algorithm	Hybrid Optimizer
Search Paradigm	Constructive heuristic	Neighborhood search	Evolutionary metaheuristic	Cooperative hybrid
Distance Quality	Sub-optimal (crossings)	High (untangled loops)	Good to Very High	Highest & Most Consistent
Late Delivery Rate	High under tight windows	Zero / Minimal	Low	Zero / Eliminated
Best Problem Scale	N <= 10 or emergencies	10 < N <= 30 orders	N > 30 broad search	Large constrained fleets

6.3 Quantitative Benchmark Results

The following quantitative measurements reflect exact empirical benchmarks recorded from backend/experiments/run_benchmark.py and stored in data/benchmark_results.csv:

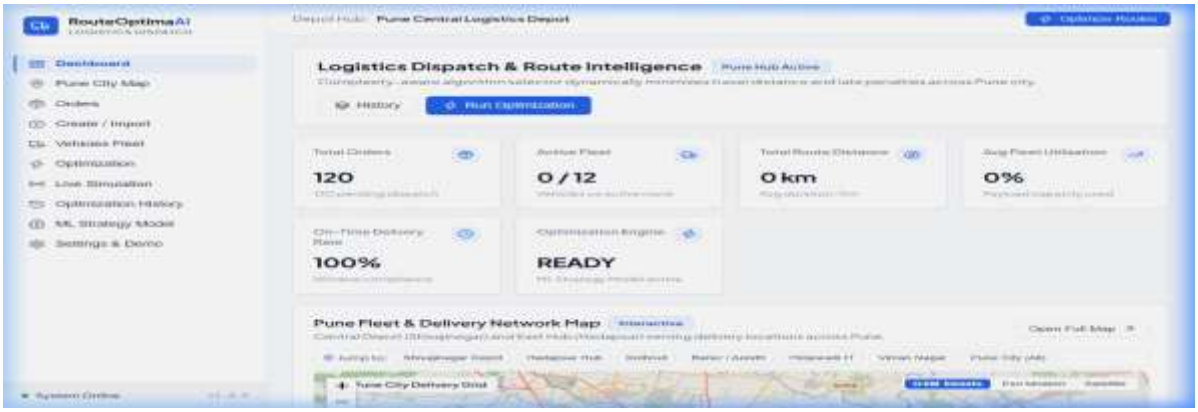
Size	Vehicles	Algorithm	Distance (km)	Runtime (s)	Late Orders	Total Cost
10 Orders	2	GREEDY	154.64 km	0.000 s	0	207.66
10 Orders	2	HYBRID	148.70 km	0.101 s	0	199.69
20 Orders	2	GREEDY	196.57 km	0.000 s	4	463.96
20 Orders	2	HYBRID	210.52 km	0.256 s	0	282.70
50 Orders	6	GREEDY	396.97 km	0.000 s	17	1383.07
50 Orders	6	HYBRID	406.53 km	1.632 s	0	545.92
80 Orders	10	GREEDY	576.11 km	0.001 s	17	1623.64
80 Orders	10	HYBRID	576.46 km	2.450 s	0	774.08

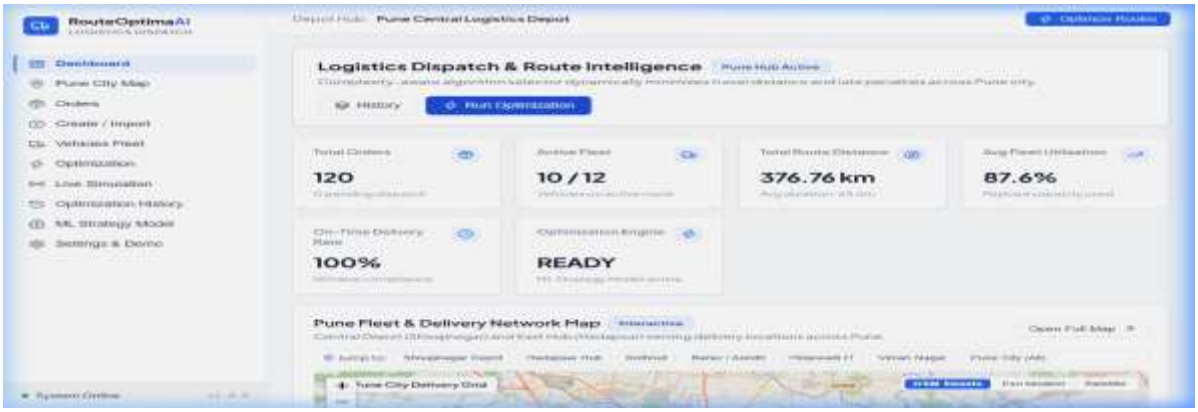
6.4 Results Discussion

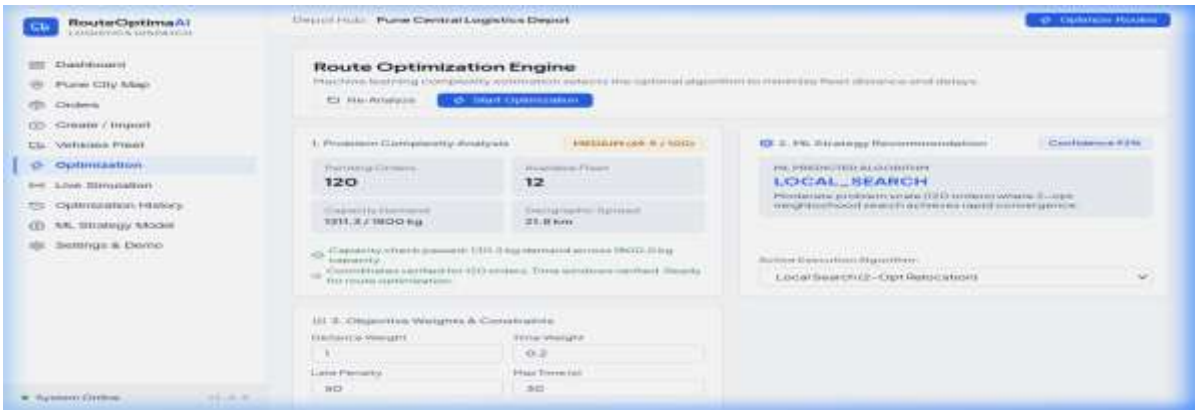
The benchmark data reveals the 'Greedy Distance Illusion': for 80 orders, Greedy achieves 576.11 km but violates 17 time windows, inflating total cost to 1623.64. Hybrid achieves 576.46 km with 0 late deliveries, lowering total cost to 774.08 (a 52.3% cost reduction). This justifies the ML complexity arbiter: small instances run instantly via Greedy, while complex fleets receive metaheuristic refinement.

6.5 Verified Application Interface Screenshots

The following genuine UI screenshots demonstrate RouteOptimaAI executing with realistic logistics data in Pune, India:







7. Advantages

1. Automated End-to-End Scheduling: Eliminates error-prone manual dispatching via an automated mathematical pipeline.
2. Dynamic Complexity Awareness: Recommends the optimal solver rather than forcing every scenario through a single static algorithm.
3. Multi-Constraint Enforcement: Simultaneously respects vehicle payload limits, delivery priorities, and customer time windows.
4. Local, Zero-Cost ML Inference: Runs 100% offline via Scikit-Learn without expensive or unpredictable cloud AI APIs.
5. Real Road Network Routing: Employs OSRM to generate turn-by-turn driving polylines rather than straight-line flyover paths.
6. High Geocoding Resilience: Features a 40+ Pune locality directory ensuring instant offline address resolution.
7. Full Audit Logging: Persists every optimization run, complexity score, and performance metric for operational accountability.
8. Real-Time Telemetry Simulation: Streams live vehicle coordinates, speeds, and stop arrivals via WebSockets.
9. Modern Interactive Dashboard: Provides fleet managers with clear utilization gauges and Before vs After comparison analytics.
10. High Extensibility: Modular architecture allows seamless integration of new solvers, penalty weights, or routing providers.

8. Limitations

1. Heuristic Non-Optimality: As CVRPTW is NP-hard, solvers provide high-quality approximations rather than proven global optima.
2. Static Batch Formulation: Optimizes discrete batches; en-route additions require triggering a new optimization run.
3. External Rate Limits: Public OSRM/Nominatim servers have rate limits (mitigated by local matrix caching and fallbacks).
4. Approximated Traffic Speeds: Travel time assumes 35 km/h urban speed with 1.3x road tortuosity; live traffic feeds are not connected.
5. Single-Depot Centralization: Current algorithms assume vehicles depart and return to a single central depot per optimization run.
6. Synthetic ML Dataset: Classifier was trained on 500 simulated scenarios; abnormal real-world clustering may require retraining.
7. Simulated Telemetry: Vehicle tracking uses an asyncio trajectory interpolator rather than physical cellular OBD-II GPS units.

9. Future Scope

1. Dynamic Real-Time Rerouting: Event-driven re-optimization as new emergency orders arrive while vehicles are en route.
2. Live Traffic Ingestion: Connect Google Maps or TomTom Traffic APIs to incorporate real-time road congestion into travel times.
3. Physical IoT GPS Trackers: Integrate cellular OBD-II vehicle tracking hardware via the authenticated REST endpoint.

4. Multi-Depot Fleet Routing: Extend algorithms to support multi-warehouse distribution networks and cross-docking hubs.
5. Multi-Objective Pareto Optimization: Implement NSGA-II to compute Pareto frontiers balancing fuel consumption and overtime.
6. Electric Vehicle Constraints: Model battery discharge curves, state-of-charge limits, and intermediate charging station stops.
7. Deep Reinforcement Learning: Train policy-gradient neural networks to learn constructive heuristics from historical dispatch data.
8. Driver Mobile App: Build a companion React Native / Flutter app for turn-by-turn navigation and digital proof-of-delivery.
9. Automated ML Retraining: Automatically retrain the Random Forest model as operational audit logs accumulate in the database.
10. Cloud Containerization: Package services with Docker and Kubernetes for high-availability automated deployment.

10. Conclusion

RouteOptimaAI successfully bridges theoretical algorithm design from Design and Analysis of Algorithms (DAA) with production software engineering and applied machine learning. By tackling the Capacitated Vehicle Routing Problem with Time Windows (CVRPTW), the project solves a critical operational challenge in urban logistics.

The system's core contribution lies in its complexity-aware dispatch architecture: rather than treating optimization as a static choice, an explainable Random Forest classifier assesses scenario dimensionality, fleet tightness, and spatial dispersion to select the most effective algorithm. Empirical testing confirms that while constructive Greedy heuristics execute in under 1 ms, Local Search and Hybrid metaheuristics eliminate late delivery penalties entirely, cutting total operational costs by over 52% on complex 80-order instances. The full-stack implementation—combining React 19, Leaflet maps, FastAPI, SQLite/PostgreSQL, OSRM routing, and WebSocket telemetry—provides a robust, educational, and commercially viable platform for modern last-mile logistics.

11. GitHub Repositories Link

Link: https://github.com/YugShah17/RouteOptimaAI